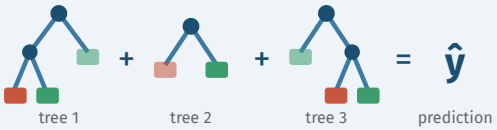
Interpretable tree sums with figsr : : CHEATSHEET

figsr fits **FIGS** (Fast Interpretable Greedy-Tree Sums; Tan et al., 2023): a model that is a **sum of a few small trees**, as accurate as much bigger tree models and still readable as plain IF-THEN rules.

Basics

Every prediction is the **sum of one leaf from each tree**. Trees are small, so the whole model fits on a slide.



At each step FIGS scores **one pool of candidate splits** and keeps the one that cuts the residual error the most:



grow a new tree from a fresh root, fit to the residuals of the others, **or**



deepen an existing tree by splitting one of its leaves.

It stops after `max_splits` splits, or when no split helps.

Install

```
install.packages("figsr")
# development version
pak::pak("bonijoao/figsr")
library(figsr)   # needs R >= 4.1
```

Data



pima_diabetes 768 patients, 8 clinical predictors and a two-class `Outcome`. The impossible zeros are already NA, ready for `na_method = "mia"`.

```
summary(pima_diabetes)
```

Any data frame works: numeric and factor predictors; a numeric outcome (regression) or a two-level factor (classification).

Missing values `na_method = "mia"`



Missingness Incorporated in Attributes. Every row is kept. Each split learns which side missing values go to, or splits on missingness itself; prediction follows the same path. Rules read `IS MISSING`.

```
figs(Outcome ~ ., pima_diabetes,
  mode = "classification",
  na_method = "mia")
```

Fit a model

THE MODEL `figs()`

```
figs(formula, data, max_splits = 10,
     max_trees = NULL, min_n = 5,
     mode = "regression",
     na_method = c("omit", "mia"))
```

`max_splits`	total splits in the whole sum: the **size knob**
`max_trees`	cap on the number of trees (`NULL` = none)
`min_n`	minimum observations in a node
`mode`	`"regression"` or `"classification"`
`na_method`	`"omit"` drops rows, `"mia"` keeps them



Regression: leaves add up to the predicted value.

```
fit <- figs(mpg ~ wt + hp + disp,
  data = mtcars, max_splits = 5)
```



Classification (two classes): leaves add up to `P(y = level 2)`, no link function.

```
figs(Outcome ~ ., pima_diabetes,
  mode = "classification")
```

Predict



predict(fit, new_data, type) returns a tibble, one row per row of `new_data`.

```
predict(fit, mtcars)
```

`"numeric"`	`.pred` (regression default)
`"class"`	`.pred_class` (classification default)
`"prob"`	`.pred_no`, `.pred_yes`: one column per level

```
predict(fit, pima_diabetes, type = "prob")
#   .pred_no .pred_yes
# 1    0.137     0.863
```

Under `"mia"`, missing values in `new_data` follow the directions learned at fit time. `predict_figs()` is the same function, under the name 'parsnip' calls.

Ensembles



bagging_figs(formula, data, n_estimators = 10, max_splits = 6) bootstrap FIGS fits, averaged by `predict()`.

```
bag <- bagging_figs(mpg ~ ., mtcars)
predict(bag, mtcars)
```

Inspect



print(fit) overview: mode, trees, splits and predictors used.



summary(fit) every tree as **IF-THEN rules** with the leaf value each path adds.

```
--- Tree 1 ---
  |-- IF Glucose <= 143 OR Glucose IS MISSING
  |   |-- IF Glucose <= 106
  |   |   `-- Leaf Value: +0.1534
  |   `-- IF Glucose >  106 OR Glucose IS MISSING
  |       `-- Leaf Value: +0.3468
  `-- IF Glucose >  143
      `-- Leaf Value: +0.7159
--- Tree 2 ---
  |-- IF BMI <= 26.31
  |   `-- Leaf Value: -0.2148
  ...
```

A prediction adds one leaf per tree: here, the Glucose leaf plus the BMI leaf.

Visualize 'ggplot2'

FIVE VIEWS `ggfigs(fit, type = ...)`



ggfigs(fit) `type = "tree"`: node-link diagram of every tree; edge width = observations, leaf color = contribution.



ggfigs(fit, "rules", data = d) each tree as a rule list, + between trees; `data` adds the outcome rate or mean.



ggfigs(fit, "waterfall", newdata = d[1,]) how one prediction is built, one bar per tree.



ggfigs(fit, "cuts", data = d) every numeric cutpoint drawn over the data it split.



ggfigs(fit, "profile", data = d) exact partial dependence for each split predictor.



interactive = TRUE HTML widget with a tooltip on every node, leaf and cut (needs 'ggiraph').

```
ggfigs(fit, interactive = TRUE)
```

`autoplot(fit, type)`: the same plots via the 'ggplot2' generic.

Plot base graphics

```
plot(fit, tree_idx = NULL, style = "scientific",
     main = NULL, tree_names = NULL)
```



`main` one title over all panels; `tree_names` renames the per-tree titles; `tree_idx` picks trees. Leaves are colored by the sign of their contribution.

Importance



figsr_importance(fit, relative = TRUE) each predictor's summed split gain, in % of the total. Under `"mia"` a split on missingness gets its own `missing(x)` row.

tidymodels

MODEL SPEC `figs_tree()`

```
library(parsnip)
spec <- figs_tree(max_splits = 8, max_trees = 3) |>
  set_mode("classification")   # engine "figsr"
fit(spec, Outcome ~ ., data = pima_diabetes)
```

Works in `workflows` with 'recipes'; `update()` changes a spec.

TUNING `dials`

```
spec <- figs_tree(max_splits = tune(),
                  max_trees = tune())
grid <- dials::grid_regular(max_splits(),
          max_trees(), levels = 4)
tune::tune_grid(spec, Outcome ~ .,
          resamples = folds, grid = grid)
```

GOOD TO KNOW

- Classification handles **two classes** only.
- `log(x)` and other transforms work in the formula; `poly(x)` is rejected at fit time.
- A factor with more than 10 levels in a node is not split on there (the subset search is exponential).
- A missing value on a branch trained without them, or an unseen factor level, errors at predict time.

Tan et al. (2023). *PNAS*, doi:10.1073/pnas.2310151122 • Updated 2026-10